PhotoMatch — Manual Técnico de Defensa

Trabajo Práctico 08: Infraestructura como Código (IaC) con Terraform

Ingeniería de Software III · Universidad Católica de Córdoba (UCC) · 2026

Alumno: Lautaro Alejo Vallino
Legajo: 2301009 · Comisión 2
Herramienta: Terraform v1.17.0

Release Oficial: `v8.0.0`
Provider: kreuzwerker/docker 4.6.0
Entorno: Preproducción efímera (:3003)

A. Introducción Conceptual

1. Paradigma Declarativo vs. Imperativo

- **Imperativo (Scripts Bash):** Se detalla paso a paso qué comandos ejecutar. Falla si un recurso ya existe o requiere condicionales defensivos complejos.
- **Declarativo (Terraform):** Se especifica el **estado deseado final**. Terraform infiere el grafo de dependencias y calcula la ruta óptima para converger hacia ese estado.

2. Estado (`terraform.tfstate`), Drift e Idempotencia

- **Archivo de Estado:** Mapea los identificadores lógicos del código HCL con los IDs físicos del proveedor Docker. Dado que contiene secretos en texto plano, está estrictamente excluido del versionado en `.gitignore`.
- **Drift (Desviación):** Desincronización entre la realidad del proveedor y el código. Se detecta con `terraform plan`.
- **Idempotencia:** La ejecución sucesiva de `terraform apply` sobre una infraestructura que ya convergió no introduce ninguna modificación adicional (`0 to add, 0 to change, 0 to destroy`).

B. Estado Inicial de PhotoMatch antes del TP8

- Solo se utilizaba Docker Compose (`compose.yml`) para QA y Producción.
- No existía módulo formal de IaC ni gestión declarativa de estado con Terraform.
- No existía un tercer ambiente efímero (**preprod**) capaz de nacer, validarse y destruirse a demanda sin afectar QA ni Producción.

C. Implementación Paso a Paso

1. **Módulo Modular en `infra/`:**
   - `versions.tf`: Fija Terraform `v1.17.0` y el proveedor Docker en `~> 4.6.0`.
   - `variables.tf`: Parametriza imágenes GHCR inmutables y credenciales con `sensitive = true`.
   - `main.tf`: Declara los 8 recursos interconectados de preproducción.
   - `outputs.tf`: Expone la URL web `http://127.0.0.1:3003`.
2. **Los 8 Recursos Declarados:**
   - `docker_network.preprod`: Red aislada `photomatch-preprod`.
   - `docker_volume.db`: Volumen de persistencia `photomatch-preprod-data`.
   - 3 recursos de imagen: `docker_image.postgres`, `backend` y `frontend` (consumiendo etiquetas SHA de GHCR).
   - 3 contenedores: base de datos PostgreSQL 16, backend Express y frontend Nginx en el puerto 3003.

3. **Resolución de Conectividad en macOS:** Se exportó `DOCKER_HOST` a partir del socket expuesto por Docker Desktop y se parametrizó un `SESSION_SECRET` con longitud válida (≥32 caracteres).

D. Decisiones de Ingeniería y Trade-offs

Decisión Tomada	Alternativa Descartada	Justificación Técnica y Trade-off
Provider Docker fijado en 4.6.0	Provider de nube comercial (AWS/Azure)	Permite aprovisionamiento profesional local reproducible sin costos de tarjeta ni límites de cuota universitaria.
Aislamiento en Puerto 3003	Reutilizar puertos de QA o PROD	Permite demostrar la convivencia pacífica simultánea de tres ambientes sin interferencias cruzadas.
Variables Sensibles `sensitive = true`	Archivo `terraform.tfvars` versionado	Evita que contraseñas y secrets queden registrados en el historial de Git o expuestos en logs de terminal.

E. Pruebas y Evidencias Reales

Evidencias Auditables en Vivo

1. **Aprovisionamiento Exitoso:** `terraform apply` -> 8 recursos creados. Endpoint `/health` verificado en `http://127.0.0.1:3003/health` (200 OK con DB).
2. **Coexistencia Simultánea:** Verificación concurrente de QA (:3001), PROD (:3002) y PREPROD (:3003) respondiendo 200 OK.
3. **Idempotencia Demostrada:** Segundo `terraform plan` -> *«No changes. Your infrastructure matches the configuration»*.
4. **Destrucción Limpia:** `terraform destroy` -> 8 recursos eliminados. QA y PROD continuaron operativos sin interrupción.
5. **Release Oficial:** `v8.0.0` publicada en GitHub.

F. Preparación para la Defensa Oral

Pregunta 1: ¿Por qué el archivo `terraform.tfstate` nunca debe subirse al repositorio de Git?

Respuesta para defender:

Porque almacena en texto plano todos los atributos de los recursos creados, incluyendo contraseñas de base de datos y claves secretas de sesión. Además, versionar el estado en Git generaría conflictos de concurrencia inmanejables si múltiples operadores aplican cambios simultáneos.

Pregunta 2: ¿Qué significa el término «Drift» en Infraestructura como Código?

Respuesta para defender:

Es la divergencia que ocurre cuando alguien modifica manualmente un recurso (ej. detiene un contenedor o borra una red en Docker) por fuera de Terraform. En el siguiente `terraform plan`, Terraform detecta esa diferencia y planifica las acciones necesarias para restaurar el estado deseado.

Pregunta 3 (Difícil): ¿Qué pasaría si dos miembros del equipo ejecutan `terraform apply` al mismo tiempo?

Respuesta para defender:

Se produciría una condición de carrera sobre el archivo de estado, provocando corrupción de datos o creación de recursos duplicados. Para prevenirlo se utiliza **State Locking**, un mecanismo que bloquea el estado durante las operaciones de escritura (en entornos locales se bloquea automáticamente; en la nube se usa DynamoDB o Consul).

G. Resumen Final de Estudio

- **IaC:** Gestión declarativa y versionada de infraestructura con grafo de dependencias automático.
- **Idempotencia:** Aplicaciones sucesivas garantizan cero cambios residuales.
- **Ciclo de vida:** `plan` (8 altas) -> `apply` (:3003 200 OK) -> `destroy` (8 bajas limpias).
- **Pitch de 2 minutos:** *«En el TP8 aplicamos Infraestructura como Código a PhotoMatch utilizando Terraform con el provider de Docker. Declaramos de forma reproducible 8 recursos en infra/: red aislada, volumen persistente, imágenes inmutables de GHCR y contenedores enlazados dinámicamente. Demostramos el ciclo de vida completo: levantamos el ambiente preprod en el puerto 3003 verificando 200 OK con base de datos, demostramos estricta idempotencia y convivencia simultánea con QA y Producción, y finalizamos con la destrucción limpia de los recursos sin afectar la operación continua del sistema.»*